AI600 Deep Learning

Assignment #2

Due: Monday March 2, 2026

Professor:	Prof. Dr. Momin Uppal
TA(s):	Muhammad Aleem Shakeel, Muhammad Usman, Muhammad Qasim Ayub, Usman Ahad
Student:	Faaiz Khan
Roll No.:	25280051

Honor Pledge:

I affirm that I have abided by the academic integrity policy. I have neither given nor received unauthorized aid on this assignment.

All work, code implementation, and data for this assignment can be found at the following public GitHub repository:

https://github.com/FaaizKhan1999/ai600-assignments/tree/main/assignment-02

Question 1: The "Quick, Draw!" Challenge

Part A: The Pancake (Width Focus)

In this part, a shallow but wide network was trained. This `pancake_model` has 2 hidden layers, with 1024 and 2048 neurons per layer respectively.
The two plots below provide the training and validation accuracy per epoch, as well as the average loss per epoch for this model.

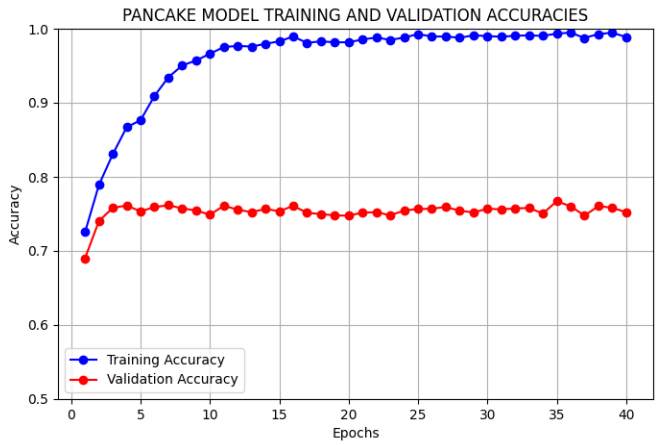

Figure 1: The per epoch training and validation accuracies for the pancake model.

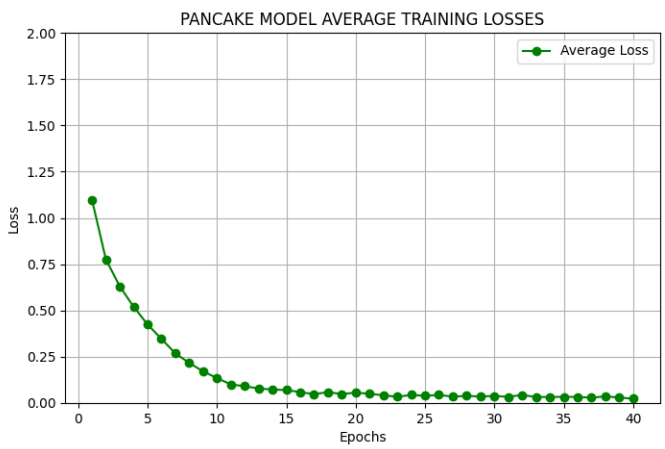

Figure 2: The per epoch average loss for the pancake model.

From the images above, it can be seen that the model's training accuracy becomes almost perfect (around 99%) as it goes through its training cycles whereas its validation accuracy plateaus very early at approximately 74%. This plateau can be seen as early as epoch 5. Therfore, it can be inferred that the model stops generalising at around that time and any learning beyond that is just overfitting. This is also reinforced by the loss curve which show that there is very minimal learning happening beyond epoch 10.

Part B: The Tower Model (Depth Focus)

In this part, a deep but narrow network was trained. This `tower_model` has 5 hidden layers with 128, 256, 512, 256, and 128 neurons per layer respectively. The two plots below provide the training and validation accuracy per epoch, as well as the average loss per epoch for this model.

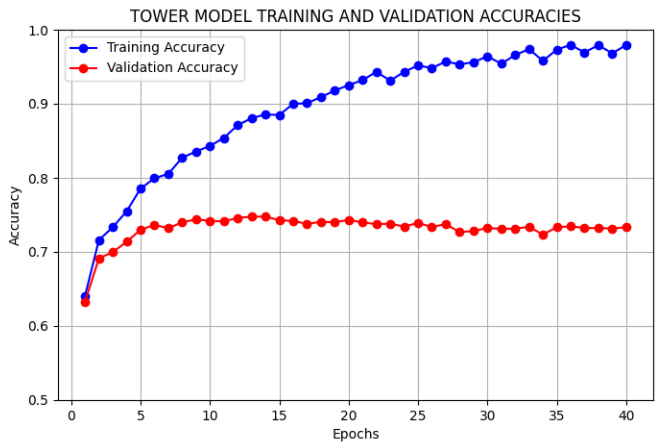

Figure 3: The per epoch training and validation accuracies for the tower model.

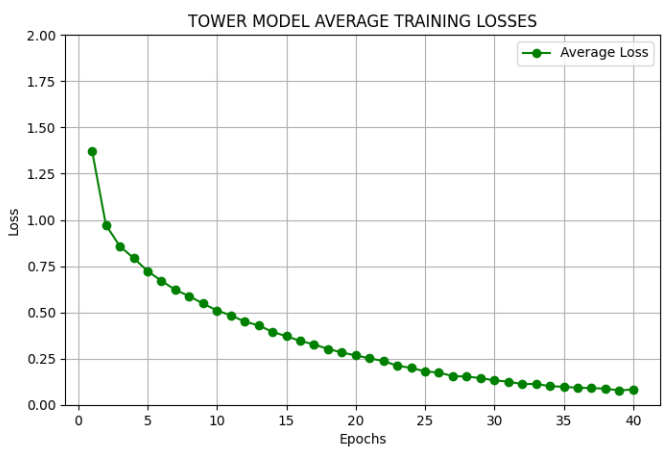

Figure 4: The per epoch average loss for the tower model.

From the images above, it can be seen that the model's training accuracy steadily increases until it reaches a very high number (around 97%) whereas — akin to the pancake model — its training accuracy plateaus at around 73%. This plateau can be seen as early as epoch 5. Once again, the inference dictates that the model stops generalising and starts overfitting. The loss curve also has a much steadier decrease compared to the pancake model.

As the network gets deeper, it may run into training difficulties in the form of vanishing gradients. While the inference from the plots cannot reliably assert that this is actually the case, the network was retrained using 1D Batch Normalisation to see if that resulted in an improvement in the network. The related plots are provided below.

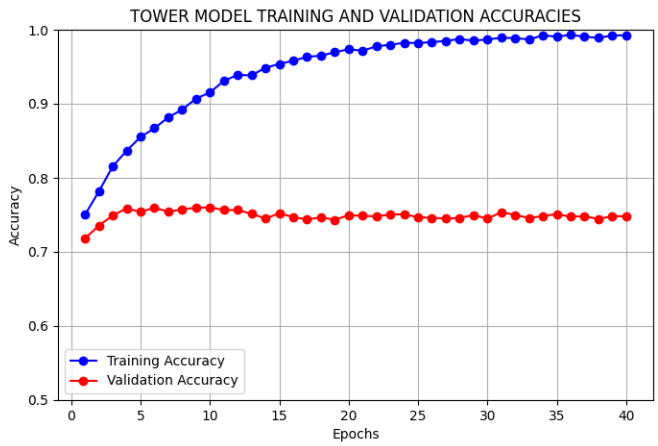

Figure 5: The per epoch training and validation accuracies for the tower model with Batch Normalisation.

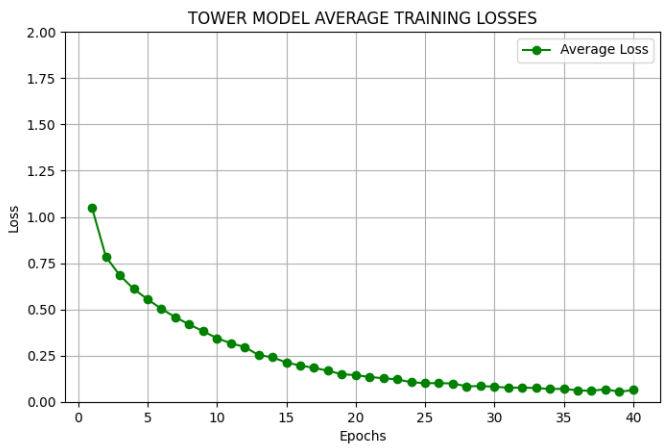

Figure 6: The per epoch average loss for the tower model with Batch Normalisation.

As can be seen in the plots, there is a noticeable difference in the training and validation accuracies of the network after Batch Normalisation is worked into the model. The training accuracy now steadily increases to nearly perfect (around 99%) as with the pancake model and the network even outperforms the pancake model, albeit marginally with a validation accuracy that plateaus at 75% but peaks at 76% at around 5 epochs. This is also the first time the effect of overfitting and the loss of generalisation is actually noticeable. As the epochs increase, the training accuracy increases but the validation accuracy starts decreasing.

Part C: The Champion (Leaderboard Submission)

To summarize the information from the first two parts, a table with the performances of each network trained over 10 epochs is provided below.

Model	**Training Accuracy (%)**	**Validation Accuracy (%)**
Pancake Model	96.86%	76.18%
Tower Model	84.28%	74.73%
Tower Model (Batch Normalization)	92.21%	76.36%

Table 1: Performance Comparison of Network Architectures

The final model is an MLP with 4 hidden layers. The layers and their components are provided below.

- **Input Layer:** 784 units.
- **Hidden Layer 1:** 512 neurons, BatchNorm, GELU activation, Dropout (0.3).
- **Hidden Layer 2:** 256 neurons, BatchNorm, GELU activation, Dropout (0.2).
- **Hidden Layer 3:** 128 neurons, BatchNorm, GELU activation, Dropout (0.1).
- **Output Layer:** 15 units.

Moving on, the different training hyperparameters/features used are also given below:

- **Label Smoothing:** Cross-entropy loss with a rate of 0.1.
- **Optimizer:** Adam with a Learning Rate of 0.002.
- **Regularization:** L1 regularization ($\lambda = 8 \times 10^{-5}$).
- **Learning Rate Scheduler:** `OneCycleLR`.
- **Activation Function:** GELU.

The model demonstrated gradual convergence over the 40 epochs.

Metric	**Score**
Training Accuracy	94.17%
Validation Accuracy	82.73%
Test Accuracy	**82.06%**

Table 2: Final Performance Metrics

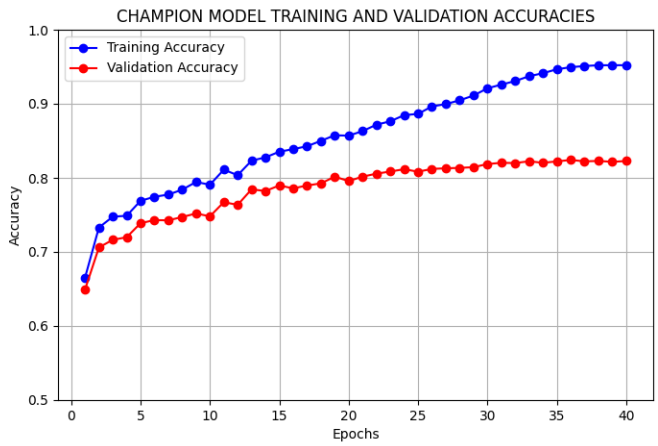

Figure 7: The per epoch training and validation accuracies for the champion model.

Finally, the network was trained over 37 epochs and contains only $569,871$ parameters, well below both thresholds.

Part D: Theoretical Analysis

1. The pancake model trained had $2,933,775$ parameters whereas the tower model had only $433,807$. Even though the difference in the performance of these models was barely noticeable (after the addition of Batch Normalisation to the tower model), there is some intuition behind preferring a deeper architecture despite what the Universal Approximation Theorem may state. Firstly, the theorem states that a single hidden layer is sufficient to approximate any continuous function. However, it does not state that the resulting model will be practical. The important thing to note here is that approximation is not the end goal, the trainability and usefulness of the model is. Secondly, images are inherently heirarchical, meaning that they are created of different components which are comprised of some basic shapes which are made up of lines and so on. Think of each layer as an abstraction that may recognise these patterns hidden inside images.

2. The confusion matrix generated by the champion model's predictions on the validation set is provided below. The main takeaway from this matrix is perhaps unsurprising. The network struggles most with differentiating between shapes that look similar or have similar features: pizza/wheel, basketball/football, clock/compass; in descending order of confusion. For almost all of these pairs, this can be chalked up to aleatoric uncertainty. In a grayscale 28×28, a pizza with 8 slices and a wheel with rims (as an example) would almost be indiscernable. This same logic could be applied to a basketball and a football (sperical objects), or a clock and a compass (circular with arms/needles).

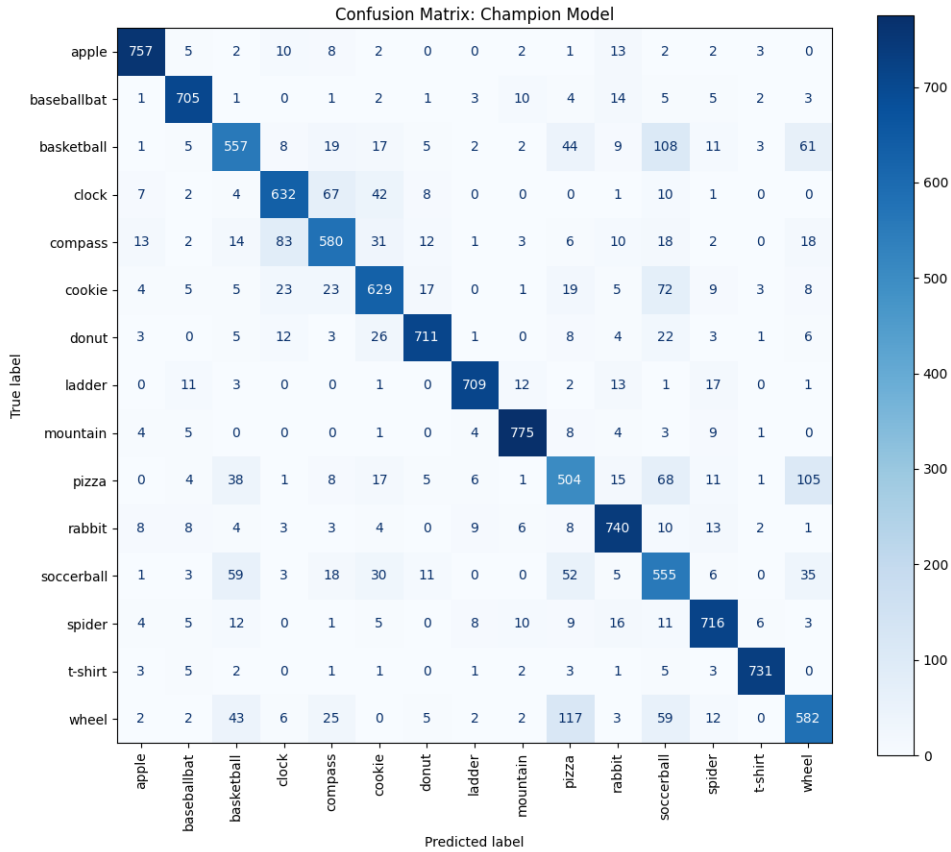

Figure 8: The champion models' confusion matrix evaluated on the validation set.

Part E: Report and Rationale When it comes to how the depth and width were chosen, the answer is simply a lot of trial and error. I initially noticed that both the pancake and the tower model exhibited huge amounts of overfitting, so Itried to look for a happy medium before settling for the architecture mentioned in Part C. The overfitting has by no means been eradicated but its effects have been mitigated by some margin.

The overfitting was combatted by making a lot of small changes to the network: these involve label smoothing, early stopping, dropout, learning rate scheduling.

The table below compares the three models and how well they performed.

Model	**Parameters**	**Training Epochs**	**Training Acc.**	**Validation Acc.**
Pancake Model	$2,933,775$	40	96.86	76.18
Tower Model	$433,807$	40	92.21	76.36
Champion Model	$569,871$	37	94.17	82.73

Table 3: Performance Comparison of Network Architectures Including the Champion Model.

GEN AI USAGE

All the content written in the report is written explicitly by me. Gen AI tools were only ever used as an aid for conceptual understanding and had no direct role in solving the questions. All research related to decision-making an understanding was also done by me.
The relevant content pertaining to the assignment was also written by myself. VS Code's Copilot was used to help provide boilerplate code instead of writing any of the logic.